Chapter 2 – Classes and Objects

(*Object-Oriented Programming*)

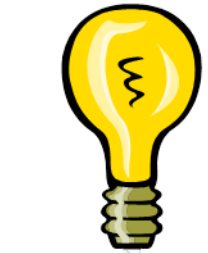
Outline

1. Review the concepts of objects and classes
2. Declaring a Class
3. Creating Objects
4. Types of class methods
5. Static members
6. Constant members
7. “this” pointer
8. Friend functions and friend classes

2.1 Review the concepts of objects and classes

Object = Data + Methods

Objects



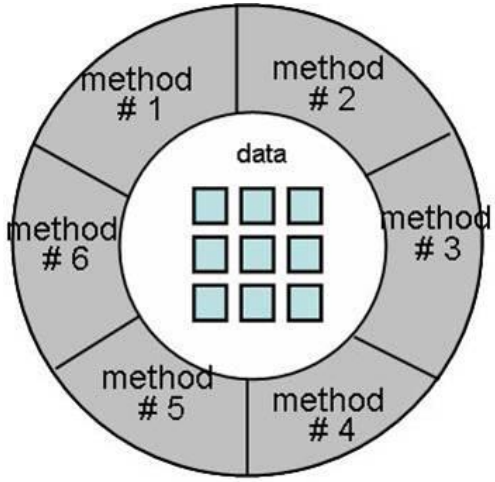
LightBulb

- **state/attributes**
  - on (true or false)
- **behavior**
  - switch on
  - switch off
  - check if on

- An object is an instance of a class.
- Each instance has its own state.
  - *Example: two different accounts will have different balances.*
- Objects are mechanisms that support data encapsulation.

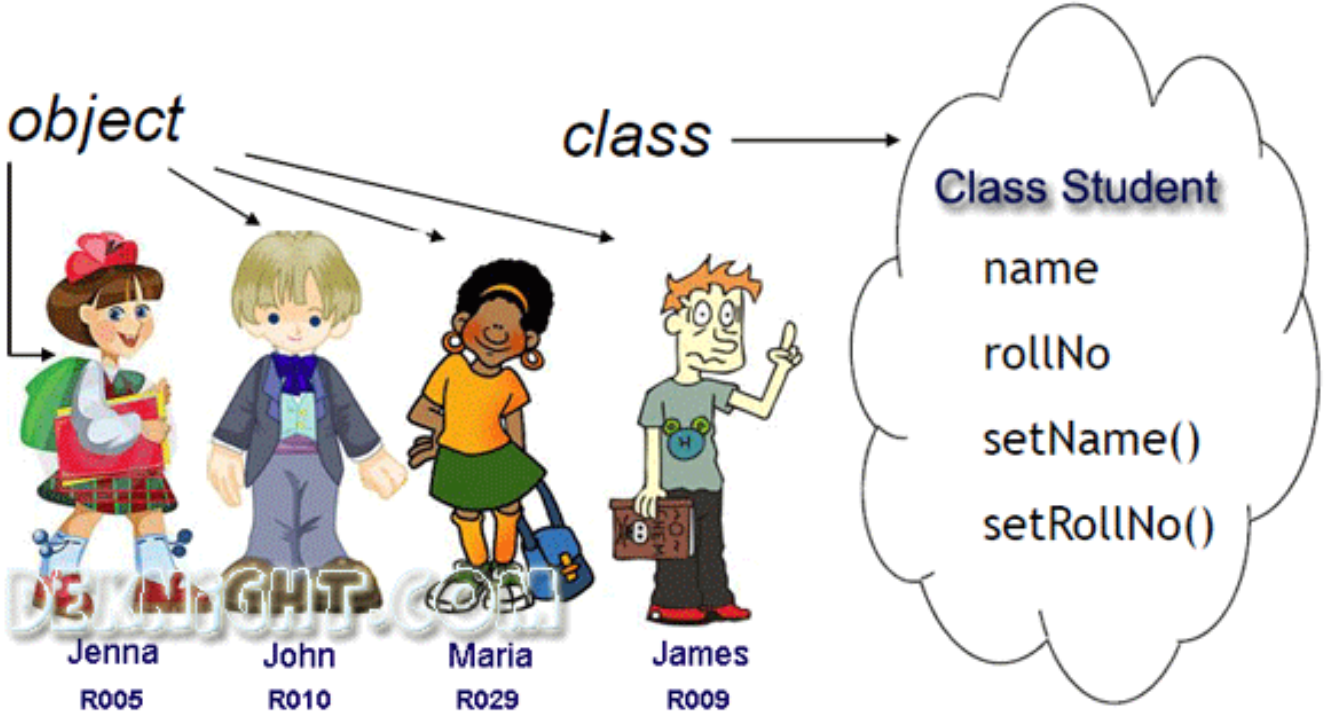
BankAccount

- **state/attributes**
  - balance
- **behavior**
  - deposit
  - withdraw
  - check balance



Classes

- Represents a collection of objects that share the same attributes and behaviors.
- A class is an implementation of ADT.



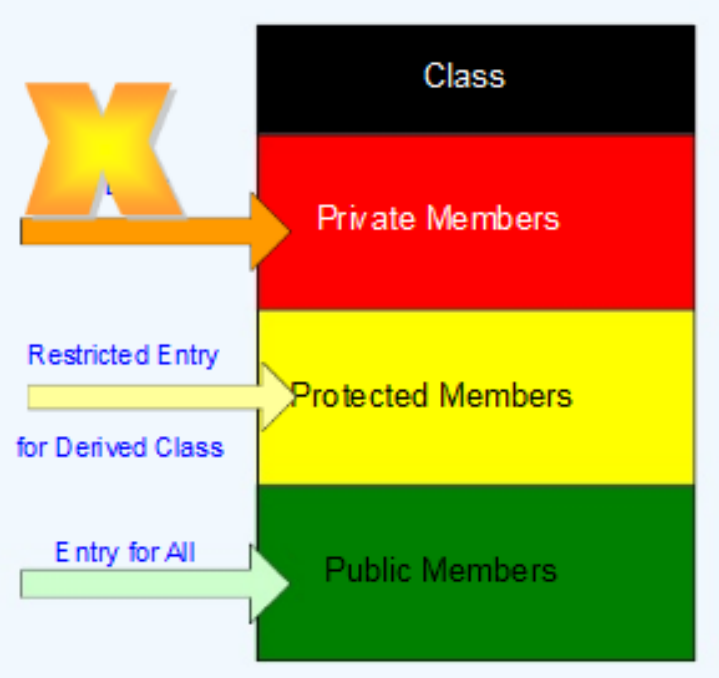
2.2 Declaring a Class

- Defines a data type.
- No memory is allocated.
- Don’t forget the semicolon after closing brace

```
class   class_name
{
    access_specifier:
        member1;
    access_specifier:
        member2;
    ...
};
```

Declaring a Class

- **access_specifier:** specifies the access level of class members
  - Public: accessible from outside the class.
  - Private: accessible only within the class
  - Protected: accessible in the class and derived classes
- Default access level depends on the programming language.
  - C++ – private, Python – public.



Declaring a Class

Class members include:

- **Data members (attributes)**

```
type  name;
double balance;
```

- **Member functions (methods)**

Declaration:

```
return_type   func(type  par1, type  par1,…)
```

Implementation:

```
return_type   class_name::func(type  par1, type  par2,…)
{
     //body of function
}
```

Declaring a Class (Interface)

```
//Rectangle.h
#pragma once

class CRectangle
{
private:
    int width, height;
public:
    void setWidth(int _width);
    int getWidth() const;
    void setHeight(int _height);
    int getHeight() const;
    int area();
};
```

Implementation of Member Functions

```
//Rectangle.cpp
#include "Rectangle.h"
int CRectangle::getHeight() const
{
    return height;
}
int CRectangle::getWidth() const
{
    return width;
}
void CRectangle::setHeight(int _height)
{
    height = _height;
}
void CRectangle::setWidth(int _width)
{
    width = _width;
}
int CRectangle::area()
{
    return width*height;
}
```

The getWidth() method is defined within the class scope

Function members can access directly any member of the class

2.3 Creating Objects

- **Object Creation Syntax**

```
class_name   object1, object2, …;
class_name   *object3;
```

Example:

```
Student   s1, s2;
CRectangle  rect1, *rect2;
```

- **Accessing class members**

```
object1.member1; //message
object1.member2;
object3→member1;
```

Example:

```
s1.input();
rect2→setWidth(2);
```

- Once an object is declared, it can access the public members of the class.
- If object is declared in the definition of a member function of the class, it can access the public and private members.

Example

```
1  //main.cpp
2  #include <iostream>
3  #include "Rectangle.h"
4  using namespace std;
5
6  int main()
7  {
8      CRectangle rect;
9      rect.height = 2;
10     rect.setHeight(3);
11     rect.setWidth(4);
12     cout<<"Dien tich: "<< rect.area()<<endl;
13     return 0;
14 }
```

Create the object rect

Accessing a member function

```
KP_CLASS, Configuration: Debug Win32 ------

main.cpp(9) : error C2248: 'CRectangle::height' : cannot access private member declared in class 'CRectangle'
```

Example

```
1  //main.cpp
2  #include <iostream>
3  #include "Rectangle.h"
4  using namespace std;
5  int main(){
6      CRectangle rect1, *rect2;
7      //rect1.height = 2;
8      rect1.setHeight(3);
9      rect1.setWidth(4);
10     cout<<"Height1: "<<rect1.getHeight()<<endl;
11     cout<<"Width1: "<<rect1.getWidth()<<endl;
12     cout<<"S1: "<< rect1.area()<<endl;
13     cout<<endl;
14     rect2 = new CRectangle();
15     rect2->setHeight(5);
16     rect2->setWidth(4);
17     cout<<"Height2: "<<rect2.getHeight()<<endl;
18     cout<<"Width2: "<<rect2->getWidth()<<endl;
19     cout<<"S2: "<< rect2->area()<<endl;
20     return 0;
21 }
```

Pointer to an object

Allocate dynamic memory for the pointer rect2

Accessing a member function

```
t from: Build
rs\user\desktop\exp_class\main.cpp(19) : error C2228: left of '.getHeight' must have class/struct/union
  type is 'CRectangle *'
  did you intend to use '->' instead?
```

Example

```cpp
//main.cpp
#include <iostream>
#include "Rectangle.h"
using namespace std;
int main(){
    CRectangle rect1, *rect2;
    //rect1.height = 2;
    rect1.setHeight(3);
    rect1.setWidth(4);
    cout<<"Height1: "<<rect1.getHeight()<<endl;
    cout<<"Width1: "<<rect1.getWidth()<<endl;
    cout<<"S1: "<< rect1.area()<<endl;
    cout<<endl;

    rect2 = new CRectangle();
    rect2->setHeight(5);
    rect2->setWidth(4);
    cout<<"Height2: "<<rect2->getHeight()<<endl;
    cout<<"Width2: "<<rect2->getWidth()<<endl;
    cout<<"S2: "<< rect2->area()<<endl;
    return 0;
}
```

```
Height1: 3
Width1: 4
S1: 12

Height2: 5
Width2: 4
S2: 20
Press any key to continue . . .
```

Creating Objects

- Once a class is properly declared and implemented, it can be used in a program
  - A program that uses/manipulates the objects of a class is called a **client** of that class
- When you declare objects of the class CRectangle, every object has its own copy of the data members (width and height).

Coding Standards

- Declare the class in the header file.

  `Rectangle.h (interface)`

- Define all member functions in the file

  `Rectangle.cpp (implementation)`

- To avoid multiple inclusion of the class declaration (`Rectangle.h`), use:

```
#ifndef _RECTANGLE_H
#define _RECTANGLE_H
//Class declaration
#endif
```

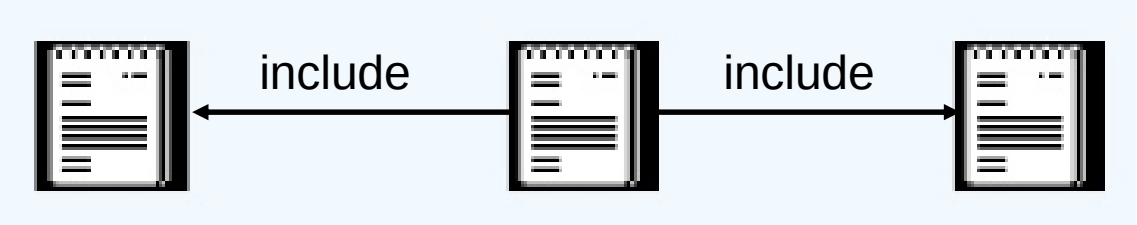
Or

```
#pragma once
```

Coding Standards

- Using the class and creating objects in the file.

```
Main.cpp (client)
```



Exercises

1. Defining and using a Class Student
   - Data members: id, name, gpa
   - Member functions: print(), input()

2. Defining and using a Class Fraction
   - Data members: numerator, denominator
   - Member functions: print(), input(), add()

2.4 Types of class methods

An object typically has four basic methods:

- **Constructor** – initialize the object
- **Destructor** – destroy the object
- **Getter** – access data
- **Setter** – update data

Constructor

- Initialize the data members of the object.
- Allocate dynamic memory for data members that are pointers.
- It is a member function of the class.
- *It is automatically called when an object is created.*

Constructor

Declaration

```
class_name(type par1, type par2,…)
```

Implementation

```
class_name::class_name(type par1, type par2,…)
{
        //body
}
```

Constructor

```cpp
//Rectangle.h
#pragma once

class CRectangle
{
private:
    int width, height;
public:
    CRectangle(int, int);
    void setWidth(int _width);
    int getWidth() const;
    void setHeight(int _height);
    int getHeight() const;
    int area();
};
```

Declare a constructor

```cpp
//Rectangle.cpp
#include "Rectangle.h"
CRectangle::CRectangle(int w, int h)
{
    width = w;
    height = h;
}
int CRectangle::getHeight() const
{
    return height;
}
int CRectangle::getWidth() const
{
    return width;
}
void CRectangle::setHeight(int _height)
{
    height = _height;
}
```

Implement a constructor

Constructor

Some characteristics of constructors:

- Have the same name as the class.
- Have no return value.
- A class can have multiple constructors (constructor overloading).
- Constructors can have parameters with default values.
- Constructors cannot be invoked like other functions.

Some Types of Constructors

- **Default Constructor**
  - Has no parameters
  - Automatically generated by the compiler if no constructor is defined in the class
- **Parameterized Constructor**
  - Has one or more parameters
  - Parameters are used to initialize the object
- **Copy Constructor**
  - Has one parameter of the same class type
  - Copies the data members from one object to another object

Invoking a Constructor

- Inovking the default constructor:

```
class_name obj;
```

- Invoking parameterized constructor:

```
class_name obj(arg1, arg2,...);
```

  - The number of arguments and their type should match the formal parameters (in the order given) of one of the constructors.
  - Otherwise, C++ uses type conversion and looks for the best match.
  - Any ambiguity leads to a compile-time error.

Example

```cpp
//main.cpp
#include <iostream>
#include "Rectangle.h"
using namespace std;
int main(){
    CRectangle rect1, *rect2;
    //rect1.height = 2;
    rect1.setHeight(3);
    rect1.setWidth(4);
    cout<<"Height1: "<<rect1.getHeight()<<endl;
    cout<<"Width1: "<<rect1.getWidth()<<endl;
    cout<<"S1: "<< rect1.area()<<endl;
    cout<<endl;

    rect2 = new CRectangle();
    rect2->setHeight(5);
    rect2->setWidth(4);
    cout<<"Height2: "<<rect2->getHeight()<<endl;
    cout<<"Width2: "<<rect2->getWidth()<<endl;
```

```
lass\main.cpp(6) : error C2512: 'CRectangle' : no appropriate default constructor available
lass\main.cpp(15) : error C2512: 'CRectangle' : no appropriate default constructor available
```

```cpp
//Rectangle.h
#pragma once

class CRectangle
{
private:
    int width, height;
public:
    CRectangle(int, int);
    void setWidth(int _width);
    int getWidth() const;
    void setHeight(int _height);
    int getHeight() const;
    int area();
};
```

Declare a two-parameter constructor

Example

```cpp
//main.cpp
#include <iostream>
#include "Rectangle.h"
using namespace std;
int main(){
    CRectangle rect1(3,4);
    //rect1.height = 2;
    cout<<"Height1: "<<rect1.getHeight()<<endl;
    cout<<"Width1: "<<rect1.getWidth()<<endl;
    cout<<"S1: "<< rect1.area()<<endl;
    cout<<endl;

    CRectangle *rect2 = new CRectangle(5,4);
    cout<<"Height2: "<<rect2->getHeight()<<endl;
    cout<<"Width2: "<<rect2->getWidth()<<endl;
    cout<<"S2: "<< rect2->area()<<endl;
    return 0;
}
```

```
Height1: 4
Width1: 3
S1: 12

Height2: 4
Width2: 5
S2: 20
Press any key to continue . . .
```

The two-parameter constructor is invoked

Example

```cpp
//Rectangle.h
#pragma once

class CRectangle
{
private:
    int width, height;
public:
    CRectangle();
    CRectangle(int, int);
    void setWidth(int _width);
    int getWidth() const;
    void setHeight(int _height);
    int getHeight() const;
    int area();
};
```

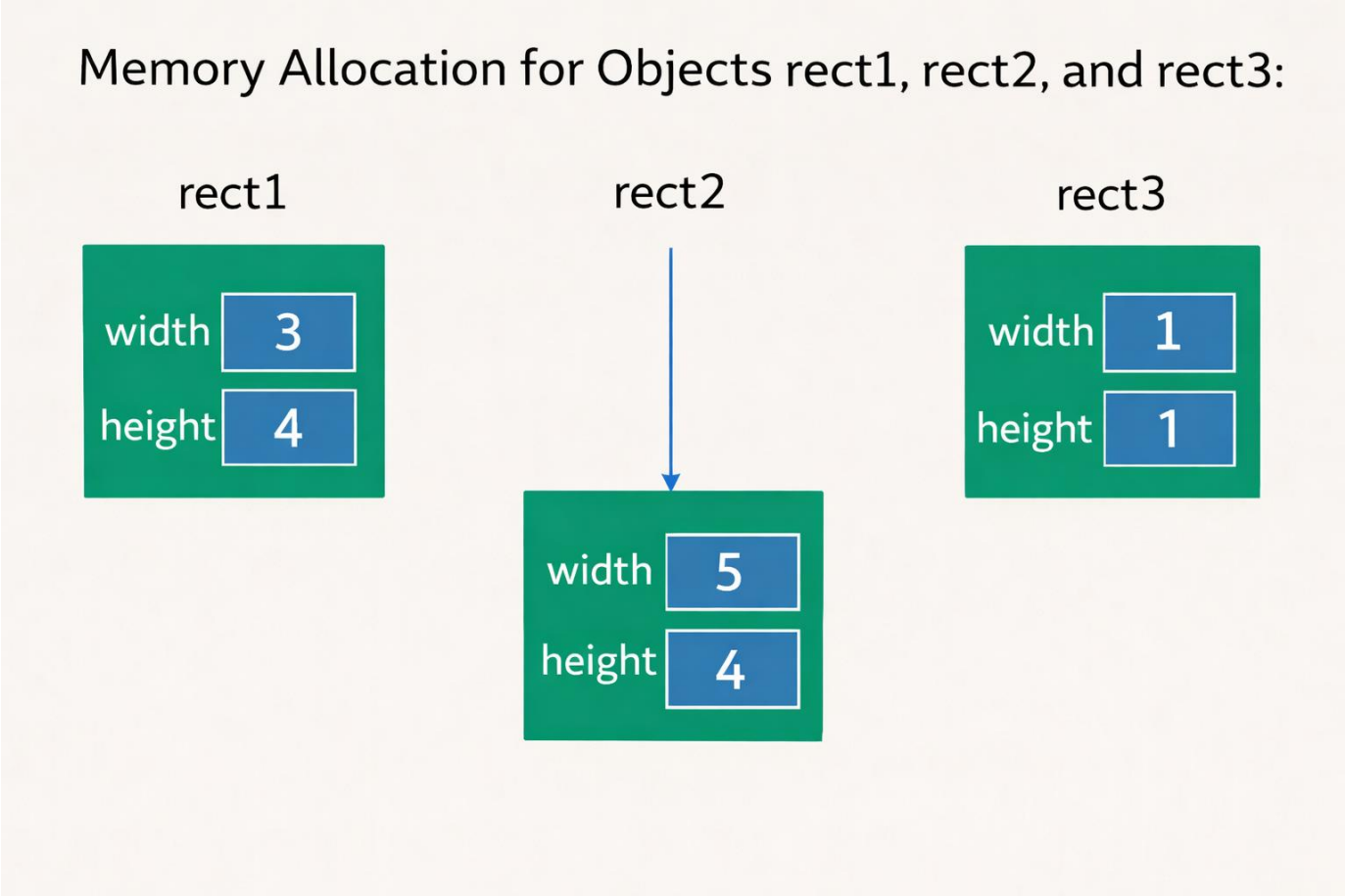
```cpp
//Rectangle.cpp
#include "Rectangle.h"
CRectangle::CRectangle()
{
    width = 1;
    height = 1;
}
CRectangle::CRectangle(int w, int h)
{
    width = w;
    height = h;
}
int CRectangle::getHeight() const
{
    return height;
}
int CRectangle::getWidth() const
{
    return width;
```

Two constructors for initializing an object

Example

```
using namespace std;
int main(){
    CRectangle rect1(3,4);
    //rect1.height = 2;
    cout<<"Height1: "<<rect1.getHeight()<<endl;
    cout<<"Width1: "<<rect1.getWidth()<<endl;
    cout<<"S1: "<< rect1.area()<<endl;
    cout<<endl;
    CRectangle *rect2 = new CRectangle(5,4);
    cout<<"Height2: "<<rect2->getHeight()<<endl;
    cout<<"Width2: "<<rect2->getWidth()<<endl;
    cout<<"S2: "<< rect2->area()<<endl;
    cout<<endl;
    CRectangle rect3;
    cout<<"Height3: "<<rect3.getHeight()<<endl;
    cout<<"Width3: "<<rect3.getWidth()<<endl;
    cout<<"S3: "<< rect3.area()<<endl;
    return 0;
}
```

Example


Memory Allocation for Objects rect1, rect2, and rect3:
rect1
width
3
height
4
rect2
width
5
height
4
rect3
width
1
height
1

Constructors and Default Parameters

- Default parameters allow a constructor to provide default values for arguments.
- Objects can be created with fewer arguments.
- This simplifies object initialization and reduces the need for multiple constructors.

Constructors and Default Parameters

```cpp
//CRectangle.h
#include <iostream>
#pragma once
class CRectangle
{
    private:
        int width, height;
    public:
        CRectangle(int w = 1, int h = 1);
};
```

```cpp
//main.cpp
#include "CRectangle.h"
int main()
{
    CRectangle r1;
    CRectangle r2(2);
    CRectangle r3(3,4);
    return 0;
}
```

```cpp
//CRectangle.cpp
#include "CRectangle.h"
CRectangle::CRectangle(int w, int h)
{
    if (w > 0)
        width = w;
    else
        width = 1;
    if (h > 0)
        height = h;
    else
        height = 1;
}
```

Copy constructor

Declaration

```
class_name(const  class_name  &arg)
```

Implementation

```
class_name::class_name(const  class_name  &arg)
{
    //body

}
```

Copy constructor

```
//Rectangle.cpp
#include "Rectangle.h"
CRectangle::CRectangle()
{
    width = 1;
    height = 1;
}
CRectangle::CRectangle(int w, int h)
{
    width = w;
    height = h;
}
CRectangle::CRectangle(const CRectangle &r)
{
    width = r.width;
    height = r.height;
}
```

Copy the data members of object r into the current object

Copy constructor

- Used to copy the data members of one object to another object.
- The copy constructor is invoked in the following situations:

1. When an object is initialized from another object

```
CRectangle  rect3 = rect1;

CRectangle  rect3(rect1);
```

2. When an object is passed to a function by value

```
void printRect(CRectangle r)
{
    // r is a copy of the object passed to the function
}
```

3. When an object is returned from a function by value

```
CRectangle createRect()
{
    CRectangle temp(3,4);
    return temp;
}
```

Destructor

- The destructor is used to:
  - Destroy an object when it is no longer used.
  - Release the memory allocated for data members.
  - Close files, delete temporary files.
  - Close network connections, database connections, etc.
- It is a member function of a class.
- It is automatically invoked whenever an object is destroyed.

Destructor

Declaration

```
~class_name()
```

Implementation

```
class_name::~class_name()
{
        //body
}
```

Destructor

Some Characteristics of a Destructor:

- It has the same name as the class and begins with the symbol ~.
- It has no return value and no parameters.

  → Each class has only one destructor.
- The compiler automatically generates a destructor if it is not explicitly defined.

Destructor

```cpp
#pragma once

class Trainee
{
private:
    int id;
    char *name;
public:
    int getID() const;
    char* getName() const;
    void setID(int _id);
    void setName(char *n);
    Trainee(void);
    Trainee(int _id, char *n);
    ~Trainee(void);
};
```

```cpp
#include <string.h>
Trainee::Trainee(void)
{
    id = 0;
    name = NULL;
}
Trainee::Trainee(int _id, char *n)
{
    id = _id;
    name = new char[strlen(n)+1];
    strcpy(name,n);
}
Trainee::~Trainee(void)
{
    if(name != NULL)
        delete []name;
}
int Trainee::getID() const
{
```

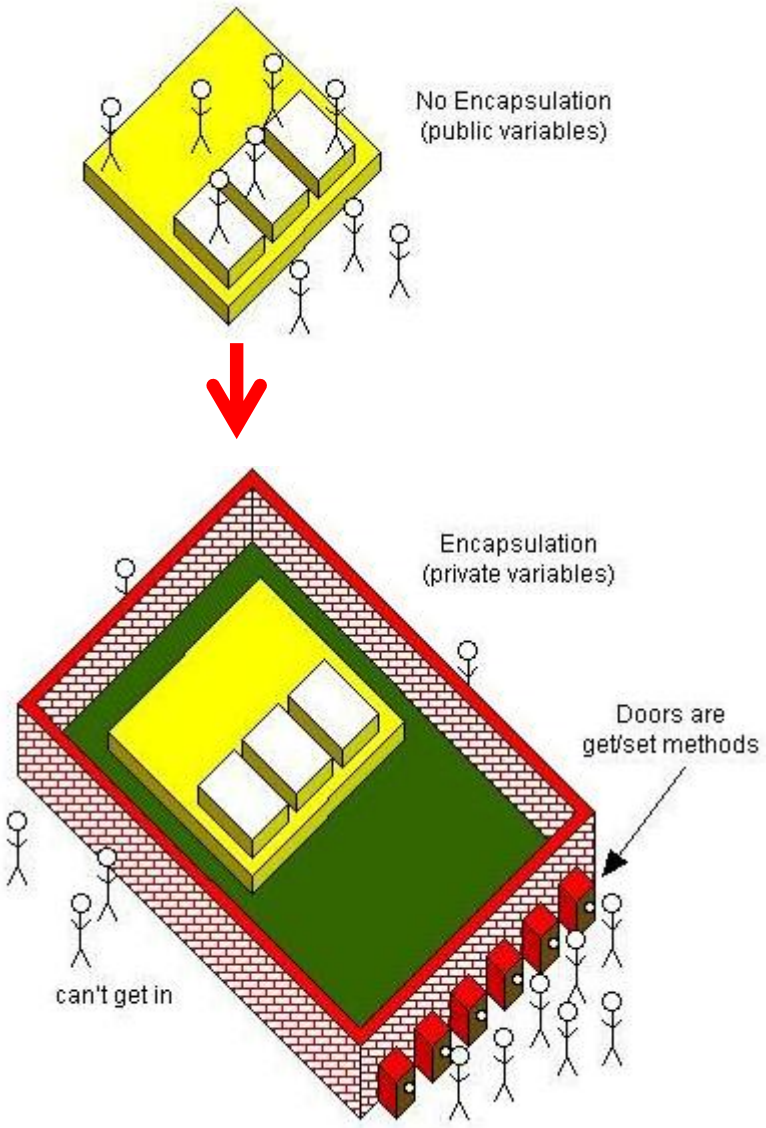
Memory for name is dynamically allocated in the constructor and freed in the destructor

Get/Set Methods

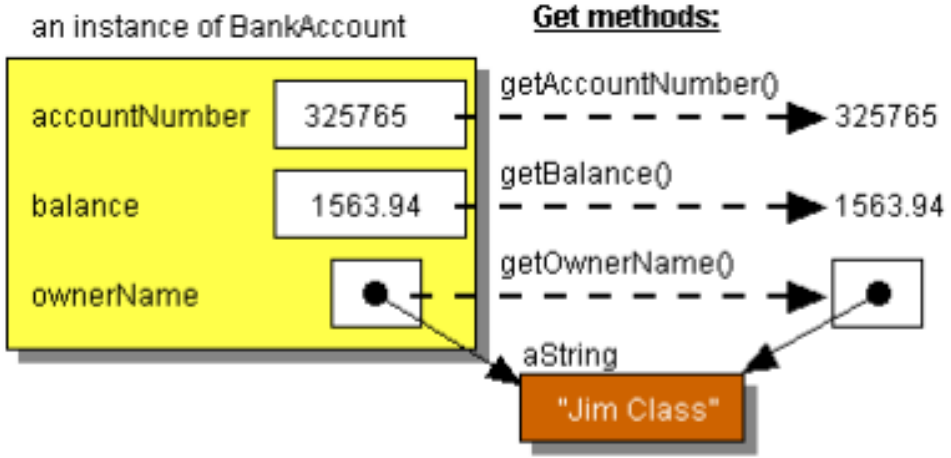
Why do we use getter and setter methods?

1. Encapsulation (Data Hiding)
   - Data members are declared private.
   - Access to the data from outside the class is controlled through public get/set methods.
2. Data Validation
   - Setter methods allow checking values before assigning them → Prevents invalid data.
3. Flexibility for Future Changes

→ Getter and setter methods protect data, validate input, and maintain encapsulation.

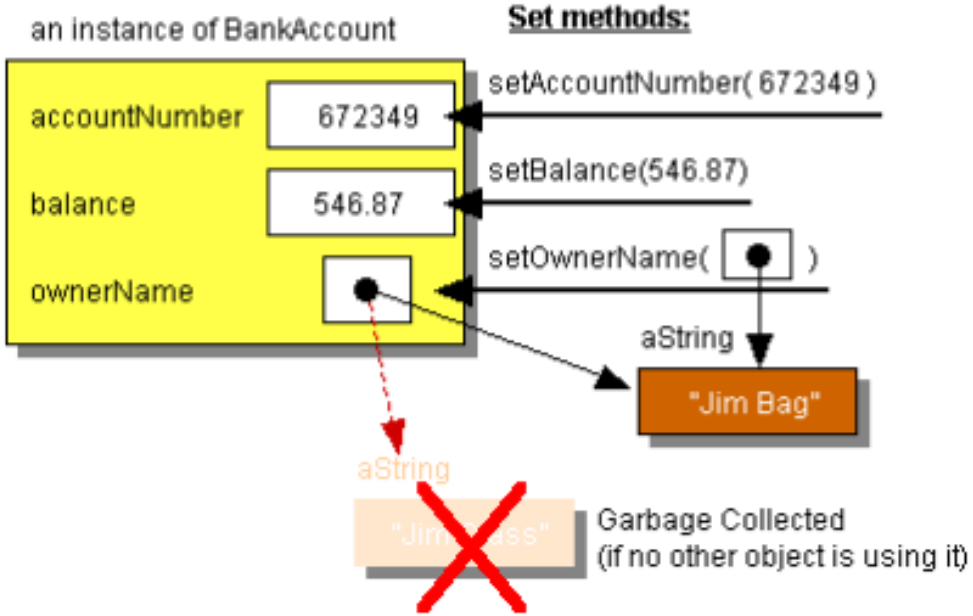


Get/Set Methods




```
void set<FieldName>(<fieldType>
    <paramName>)
{
    <fieldName> = <paramName>;
}
```

```
<fieldType> get<FieldName>() {
        return <fieldName>;
}
```



Get/Set Methods

```
int CRectangle::getWidth() const
{
    return width;
}
```

```
void CRectangle::setWidth(int _width)
{
    if(_width > 0)
        width = _width;
    else
        cout<<"Width is invalid"<<endl;
}
```

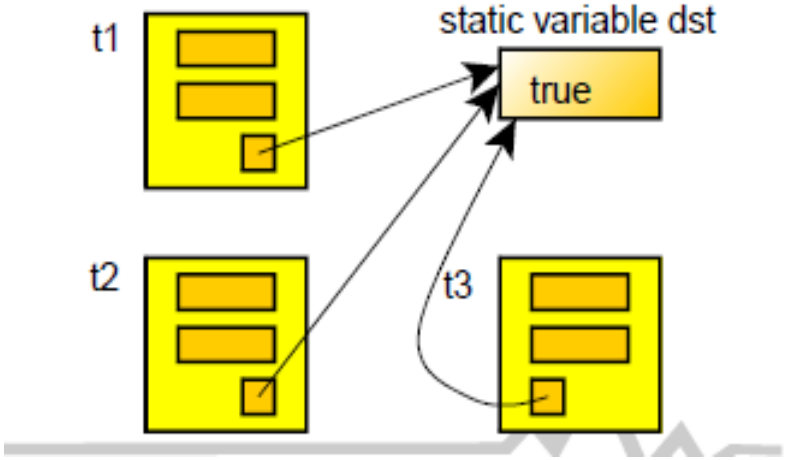
2.5 Static Members

1. Static Data Members

- The value of a static data member is shared by all objects of the class.

  → All objects of the class use the same variable.
- Static data is allocated memory only once.

  → It is not duplicated for each object.
- It can be accessed through the class name or through an object.
- It must be defined and initialized outside the class.



Static Members

Static Data Members

Declaration:

```
static  datatype  var;
static  int count;
```

Initialization:

```
datatype class_name::var = value;
```

Access through an object:

```
CRectangle   a;
a.count = 0;
```

Access through the class:

```
CRectangle::count = 1;
```

Static Members

2. Static Methods

- Shared by all objects of the class.
- Can be called without creating an object.
- Can only access static members of the class.
- Non-static member functions can access static data members.

Declaration:

```
static return_type function_name(parameter_list);
```

Static Members

```
class Trainee
{
private:
    int id;
    char *name;
    static int totalTrainees;
public:
    Trainee(void);
    Trainee(int _id, char *n);
    static int getTotalTrainees();
    ~Trainee(void);
    int getID() const;
    char* getName() const;
    void setID(int _id);
    void setName(char *n);

};
```

```
int Trainee::totalTrainees = 0;
Trainee::Trainee(int _id, char *n)
{
    id = _id;
    name = new char[strlen(n)+1];
    strcpy(name,n);
    totalTrainees++;
}
int Trainee::getTotalTrainees()
{
    return totalTrainees;
}
Trainee::~Trainee(void)
{
    if(name != NULL)
        delete []name;
    totalTrainees--;
}
```

declare and implement the static method

Initialize the Static Data Member

Static Members

```
1  #include <iostream>
2  #include "Trainee.h"
3  using namespace std;
4
5  int main()
6  {
7      Trainee *t1, *t2;
8      cout<<"So luong Trainee hien co: "<<Trainee::getTotalTrainees()<<endl;
9      t1 = new Trainee(1,"Lena");
10     cout<<"So luong Trainee hien co: "<<Trainee::getTotalTrainees()<<endl;
11     t2 = new Trainee(2,"Anna");
12     cout<<"So luong Trainee hien co: "<<Trainee::getTotalTrainees()<<endl;
13     delete t1;
14     cout<<"So luong Trainee hien co: "<<Trainee::getTotalTrainees()<<endl;
15     delete t2;
16     cout<<"So luong Trainee hien co: "<<Trainee::getTotalTrainees()<<endl;
17     return 0;
18 }
```

```
So luong Trainee hien co: 0
So luong Trainee hien co: 1
So luong Trainee hien co: 2
So luong Trainee hien co: 1
So luong Trainee hien co: 0
Press any key to continue . . .
```

2.6 Constant members

1. Const Data Members

- A const data member cannot change its value during the lifetime of an object.
- Different objects can have different values for their const data members.
- The value must be initialized in the constructor.

Declaration

```
const datatype cdata1, cdata2;
```

Initialization

```
class_name::class_name(parameters):cdata1(value),cdata2(value), ...
{
    //body
}
```

Constant members

```
class Trainee_List
{
private:
    const int size;
public:
    Trainee_List(int s);
    ~Trainee_List(void);
    int getSize() const;
};
```

```
#include "Trainee_List.h"

Trainee_List::Trainee_List(int s):size(s)
{
    //.....
}

int Trainee_List::getSize() const
{
    return size;
}
```

Initialization of
const Data Member

```
int main()
{
    //demo constant data member of class
    Trainee_List list1(100);
    Trainee_List list2(150);
    cout<<"List1 has "<<list1.getSize()<<" trainees\n";
    cout<<"List2 has "<<list2.getSize()<<" trainees\n";

```

Constant members

2. Const Member Functions

- Prevents modification of data members inside the member function.
- Getter methods are usually declared with const.

Declaration

```
return_type func_name(parameter_list) const;
```

Implementation

```
return_type func_name(parameter_list) const
{
    //body
}
```

Constant members

```
29 int Trainee::getID() const
30 {
31     id = id + 1;
32     return id;
33 }
34 char* Trainee::getName() const
35 {
36     return name;
37 }
38 void Trainee::setID(int _id)
39 {
40     id = _id;
41 }
42 void Trainee::setName(char *n)
```

char *Trainee::getName(void) const

```
x86)\microsoft visual studio 8\vc\include\string.h(74) : see declaration of 'strcpy'
: hanh so 1\exp2\trainee.cpp(31) : error C2166: l-value specifies const object
```

Constant object

- A const object cannot modify the values of its data members.
- A const object can only call static member functions or const member functions.
- Declaration: `const class_name obj;`
- Practical Examples:
  - Configuration objects (e.g., application version, system limits) should not change during program execution.
  - Objects representing fixed values such as physical constants or geometry parameters.
  - Function parameters that must not be modified.

```
void printRectangle(const CRectangle& r) {
    cout << r.getWidth() << " " << r.getHeight();
}
```

Constant object

```
5  int main()
6  {
7      //demo const object
8      const Trainee cTrainee(3,"Maria");
9
10     cTrainee.setID(4); //thay doi gia tri thanh vien du lieu --> error
11     cTrainee.printStandard(); //non-const function - error
12
13     cTrainee.getTotalTrainees(); //static function - ok
14     cout<<"Name: "<<cTrainee.getName()<<endl; //const function- ok
15
16     //demo phuong thuc static
17     Trainee *t1, *t2;
18     cout<<"So luong Trainee hien co: "<<Trainee::getTotalTrainees()<<endl;
19     t1 = new Trainee(1,"Lena");
20     cout<<"So luong Trainee hien co: "<<Trainee::getTotalTrainees()<<endl;
```

```
nh so 1\exp2\main.cpp(9) : error C2662: 'Trainee::setID' : cannot convert 'this' pointer from 'const Trainee' to 'Trainee &'
ifiers
nh so 1\exp2\main.cpp(10) : error C2662: 'Trainee::printStandard' : cannot convert 'this' pointer from 'const Trainee' to
```

2.7 “this” pointer

- Allows an object to refer to itself using the keyword this.
- Implicit first argument on non-static member function call to the object.
- Implicitly referencing data members and functions.
- The this pointer points to the current object that is calling the member function.
- The this pointer cannot be used in static member functions because static functions are not associated with a specific object.

Using the “this” pointer

1. To avoid ambiguity between data members and function parameters that have the same name.

```
#pragma once
#include <iostream>
using namespace std;
class Circle
{
private:
    float radius;
public:
    Circle(float radius = 1);
    ~Circle(void);
    float getRadius() const;
    void setRadius(float radius);
};
```

```
float Circle::getRadius() const
{
    return this->radius;
    //return (*this).radius;
    //return radius;
}
void Circle::setRadius(float radius)
{
    if (radius > 0)
        this->radius = radius;
    else
        cout<<"Radius is invalid"<<endl;
}
```

The three statements are equivalent.

Using the “this” pointer

2. Methods returning objects

```
#include <iostream>
using namespace std;
class Circle
{
private:
    float radius;
public:
    Circle(float radius = 1);
    ~Circle(void);
    float getRadius() const;
    void setRadius(float radius);
    Circle scaleUp1(int iTimes);
};
```

```
Circle Circle::scaleUp1(int iTimes)
{
    radius = radius*iTimes;
    return (*this);
}

int main()
{
    Circle c;
    c.setRadius(2);
    cout<<"Radius of c: "<<c.getRadius()<<endl;
    c.scaleUp1(2);
    cout<<"Radius of c: "<<c.getRadius()<<endl;
    return 0;
}
```

Using the “this” pointer

3. Methods returning a reference to an object

```
#include <iostream>
using namespace std;
class Circle
{
private:
    float radius;
public:
    Circle(float radius = 1);
    ~Circle(void);
    float getRadius() const;
    void setRadius(float radius);
    Circle scaleUp1(int iTimes);
    Circle& scaleUp2(int iTimes);
};
```

```
Circle& Circle::scaleUp2(int iTimes)
{
    radius = radius*iTimes;
    return (*this);
}

int main()
{
    Circle c(3);
    //method chaining
    //c.scaleUp1(2).setRadius(5);
    c.scaleUp2(2).setRadius(5);
    cout<<"Radius of c: "<<c.getRadius()<<endl;
    return 0;
}
```

2.8 friend functions and friend classes

- Data declared as private in a class cannot be accessed from outside the class.
- However, in some situations, external functions or classes may need to access private data.

→ C++ provides an exception: friend

friend functions and friend classes

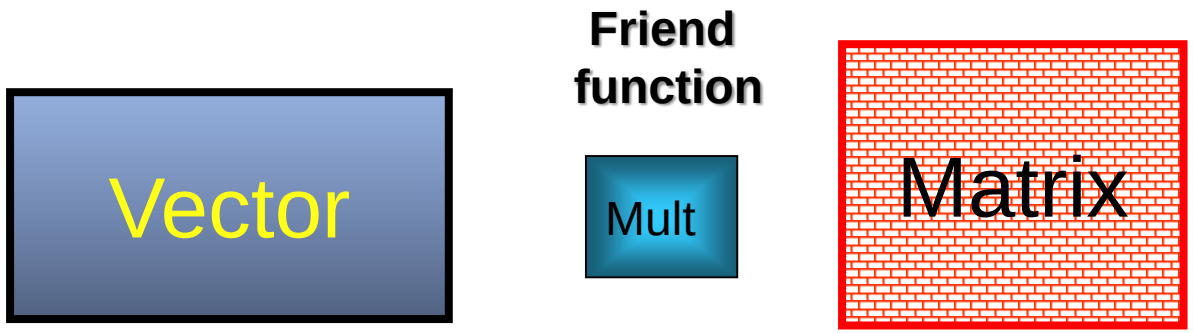
- friend functions and friend classes can access private and protected members of another class.
- Properties of friendship
  - Friendship is granted, not taken.
  - Not symmetric (if B a friend of A, A not necessarily a friend of B).
  - Not transitive (if A a friend of B, B a friend of C, A not necessarily a friend of C).

friend functions

- A friend function of a class is not a member function of that class.
- Its prototype is declared inside the class using the keyword friend.
- The function is defined outside the class scope.
- It is called like a non-member function.

Declaration:

```
friend return_type func_name(parameter_list);
```



friend functions

Types of friendship:

- A non-member (global) function can be a friend of a class.
- A member function of one class can be a friend of another class.
- A function can be a friend of multiple classes.

Using friend functions in the following cases:

- When a function needs to access private and protected members of classes.
- To implement input/output functions.
- To overload some operators.

friend functions

```cpp
#pragma once

class Point
{
private:
    int x;
    int y;
public:
    Point(int x = 0, int y = 0);
    friend bool same(Point p1, Point p2);
    ~Point(void);
};
```

```cpp
#include <iostream>
#include "Point.h"
using namespace std;

//dinh nghia ham same ben ngoai pham vi lop Point
bool same(Point p1, Point p2)
{
    return ((p1.x == p2.x) && (p1.y == p2.y));
}
int main()
{
    Point p1(3,0), p2(3),p3;
    if(same(p1,p2)) cout<<"p1 trung voi p2"<<endl;
    else  cout<<"p1 khac p2"<<endl;
    if(same(p1,p3))  cout<<"p1 trung voi p3"<<endl;
    else  cout<<"p1 khac p3"<<endl;
}
```

bool same(Point p1, Point p2)

friend functions

```
class Matrix;
class Vector {
    friend Vector multiply(const Matrix&, const Vector&);
    //...
};

class Matrix {
    friend Vector multiply(const Matrix&, const Vector&);
    //...
};

Vector multiply(const Matrix& ml, const Vector& vl) {
    //...
};
```

friend classes

- If class A is declared as a friend of class B, then all member functions of class A can access the private and protected members of class B.

```
class Point {
  ...
  friend class Line;
};
```

friend classes

```
#include <iostream>
using namespace std;

class CSquare;

class CRectangle {
    int width, height;
  public:
    int area ()
      {return (width * height);}
    void convert (CSquare a);
};

class CSquare {
  private:
    int side;
  public:
    void set_side (int a)
      {side=a;}
    friend class CRectangle;
};

void CRectangle::convert (CSquare a) {
  width = a.side;
  height = a.side;
}
```

```
int main () {
  CSquare sqr;
  CRectangle rect;
  sqr.set_side(4);
  rect.convert(sqr);
  cout << rect.area();
  return 0;
}
```

Note

- Friend functions or friend classes break encapsulation and weaken data protection.
- They should only be used when truly necessary.

Exercises

Create a class Circle including:

- Attributes
  - Center coordinates: x, y
  - Radius: radius
  - Constant: pi
  - count: a static variable used to store the number of existing objects
- Methods
  - Constructor with three parameters x, y, radius with default values 0, 0, 1 respectively
  - get/set methods corresponding to the class attributes
  - getCount(): a static method that returns the value of count
  - area(): computes the area of the circle
  - moveRight(): moves the circle to the right
- Friend Function
  - compare(): compares two circles based on their radius.